



amosproj / amos2026ss02-automated-thematic-analysis



<> Code

Issues 52

Pull requests 2

Discussions

Actions

Projects 1

Build Documentation

Edit

New page

[Jump to bottom](#)

julianlucawendlandt-jpg edited this page now · [1 revision](#)

Build & Deploy

Prerequisites

- Docker & Docker Compose v2
- (Optional, for local dev) Python 3.11+, [uv](#) or pip, PostgreSQL 16

Quick Start (Docker — recommended)

```
git clone <repository-url>
cd amos2026ss02-automated-thematic-analysis
```



Linux / macOS / Git Bash

```
chmod +x setup.sh && ./setup.sh
```

Windows PowerShell

```
.\setup.ps1
```

The bootstrap script verifies Docker is running, creates Backend/.env from the template if needed, builds images, starts the stack, and waits for the health check.

After startup, set your LLM API key in Backend/.env (LLM_API_KEY). This is required for all analysis features.

Services:

URL	Description
http://localhost:8000	API server
http://localhost:8000/docs	Swagger UI

Run ./setup.sh --help Or Get-Help .\setup.ps1 for all options.

Local Development (no Docker for the API)

```
cd Backend
cp .env.example .env          # edit with your DB credentials
```



Start only Postgres via Docker

```
docker compose up db -d
```

Install dependencies

```
uv sync          # or: pip install -e .
```

Run dev server

```
uvicorn app.main:app --reload # http://localhost:8000
```

Docker Image Targets

The Compose file defines two service profiles:

- **api** — runtime target. Production-style image, no test tooling.
- **api-test** — test target. Includes `pytest` and `pytest-cov`.

Running Tests

```
docker compose --profile test run --rm api-test \
  pytest --cov=app --cov-report=term-missing --cov-report=html
```



Coverage report is written to `Backend/htmlcov/index.html`.

Environment Variables

All configuration is driven by `Backend/.env`. See `Backend/.env.example` for the full

reference — every variable the app reads is documented there.

Manual Docker Setup (fallback)

If the bootstrap script doesn't work for your environment:

```
cd Backend
cp .env.example .env
# edit .env
docker compose up --build
```

API Response Format

Every endpoint returns a standard envelope:

```
{ "success": true,  "data": { ... }, "error": null, "meta": null }
{ "success": false, "data": null,    "error": "...", "meta": { "detail": "..." }
}
```

Project Structure

```
Backend/app/
├─ main.py           # App factory + lifespan
├─ config.py         # Pydantic Settings (.env)
├─ database.py       # Async SQLAlchemy engine + session
├─ dependencies.py   # FastAPI dependency injection
├─ exceptions.py     # Custom exceptions + handlers
├─ logging_config.py # Loguru setup
├─ middleware.py     # RequestId, logging, CORS, GZip
├─ models/           # SQLAlchemy ORM models
├─ routers/          # FastAPI APIRouters
├─ schemas/          # Pydantic request/response schemas
└─ services/         # Business logic layer
```

Build & Deploy

Prerequisites

- Docker & Docker Compose v2
- (Optional, for local dev) Python 3.11+, [uv](https://docs.astral.sh/uv/) or pip, PostgreSQL 16

Quick Start (Docker — recommended)

```
git clone <repository-url>
cd amos2026ss02-automated-thematic-analysis

# Linux / macOS / Git Bash
chmod +x setup.sh && ./setup.sh

# Windows PowerShell
.\setup.ps1
```

The bootstrap script verifies Docker is running, creates `Backend/.env` from the template if needed, builds images, starts the stack, and waits for the health check.

After startup, set your LLM API key in `Backend/.env` (`LLM_API_KEY`). This is required for all analysis features.

Services:

URL	Description
http://localhost:8000	API server
http://localhost:8000/docs	Swagger UI

Common Commands

Action	Linux / macOS	Windows PowerShell
Start stack	<code>./setup.sh</code>	<code>.\setup.ps1</code>
Start (foreground logs)	<code>./setup.sh -f</code>	<code>.\setup.ps1 -Foreground</code>
Run tests	<code>./setup.sh --test</code>	<code>.\setup.ps1 -Test</code>
Stop stack	<code>./setup.sh --down</code>	<code>.\setup.ps1 -Down</code>
Stop + delete volumes	<code>./setup.sh --down-volumes</code>	<code>.\setup.ps1 -DownVolumes</code>

Run `./setup.sh --help` or `Get-Help .\setup.ps1` for all options.

Local Development (no Docker for the API)

```
cd Backend
cp .env.example .env # edit with your DB credentials
```

```
cp .env.example .env # edit with your db credentials

# Start only Postgres via Docker
docker compose up db -d

# Install dependencies
uv sync # or: pip install -e .

# Run dev server
uvicorn app.main:app --reload # http://localhost:8000
```

Docker Image Targets

The Compose file defines two service profiles:

- **api** — runtime target. Production-style image, no test tooling.
- **api-test** — test target. Includes `pytest` and `pytest-cov`.

Running Tests

```
docker compose --profile test run --rm api-test \
  pytest --cov=app --cov-report=term-missing --cov-report=html
```



Coverage report is written to `Backend/htmlcov/index.html`.

Environment Variables

All configuration is driven by `Backend/.env`. See `Backend/.env.example` for the full reference — every variable the app reads is documented there.

Manual Docker Setup (fallback)

If the bootstrap script doesn't work for your environment:

```
cd Backend
# edit .env
docker compose up --build
```



API Response Format

Every endpoint returns a standard envelope:

```
{ "success": true, "data": { ... }, "error": null, "meta": null }
{ "success": false, "data": null, "error": "...", "meta": { "detail": "..." } }
```



Project Structure

```
Backend/app/
├─ main.py           # App factory + lifespan
├─ config.py         # Pydantic Settings (.env)
├─ database.py       # Async SQLAlchemy engine + session
├─ dependencies.py   # FastAPI dependency injection
├─ exceptions.py     # Custom exceptions + handlers
├─ logging_config.py # Loguru setup
├─ middleware.py     # RequestId, logging, CORS, GZip
├─ models/          # SQLAlchemy ORM models
├─ routers/         # FastAPI API Routers
├─ schemas/         # Pydantic request/response schemas
└─ services/        # Business logic layer
```



+ Add a custom footer

▼ Pages 13

Find a page or section...

▼ Home

▼ Backend Routes

▼ Backend Software Architecture and Data Model Documentation

▼ Build & Deploy

▼ Build Documentation

Build & Deploy

Prerequisites

Quick Start (Docker — recommended)

Linux / macOS / Git Bash

Windows PowerShell

Local Development (no Docker for the API)

Start only Postgres via Docker

Install dependencies

Run dev server

- Quick Start (Docker — recommended)
- Common Commands
- Local Development (no Docker for the API)
- Docker Image Targets
- Running Tests
- Environment Variables
- Manual Docker Setup (fallback)
- API Response Format
- Project Structure

▼ [codebook generation](#)

▼ [csv codebook standard](#)

▼ [demographic pipeline](#)

▼ [Frontend and Routing](#)

▼ [ingestion pipeline](#)

▼ [LLM cluster documentation](#)

▼ [theme data type](#)

▼ [theme graph](#)

+ Add a custom sidebar

Clone this wiki locally

https://github.com/amosproj/amos2026ss02-automated-thematic-analysis.wiki.git

